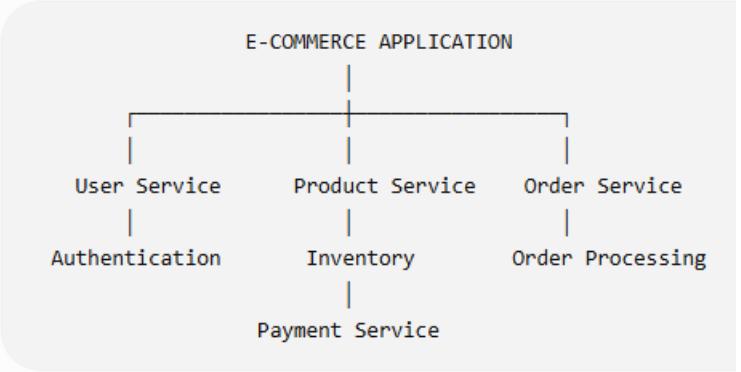
What are microservices why do we need them?

1. What are Microservices?

- **Microservices** is an **architectural style** where an application is structured as a **collection of small autonomous services.**
- The important idea is to **break one large application into multiple smaller services, with each service responsible for a specific business domain/functionality**
- Each of these can be developed and deployed as a **standalone services**



2. Services are Organized Around Business Domains

- This is one of the most important concepts
- Instead of thinking “Let’s split our code into small pieces
- We think “Let’s **identify business capabilities/domains** and create services around them”

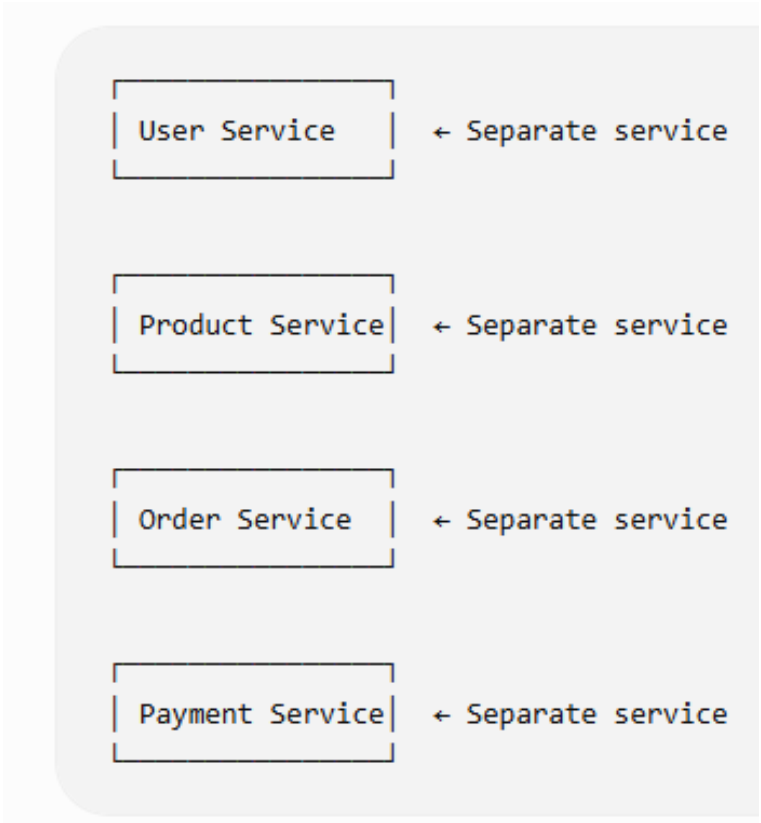
For an e-commerce system:

Business Domain	Possible Microservice
Users & Authentication	User Service
Products	Product Service
Inventory	Inventory Service
Orders	Order Service
Payments	Payment Service

The exact boundaries depend on the application.

3. Each Microservices runs as a separate process

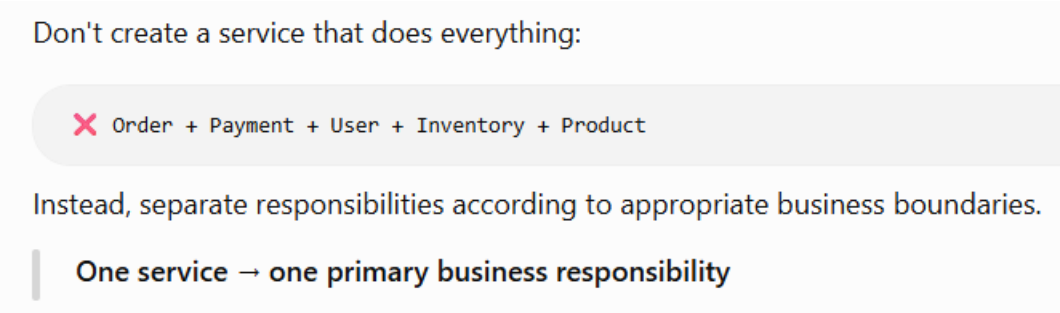
- Unlike a traditional monolithic application, where everything is part of one application process
- Microservices are designed as **separate services/processes**
- These services need to communicate with each other to accomplish business goals



4. Core Principles of Microservices

Principle 1 – Single Responsibility

- Each microservice should have **one primary responsibility** and should so that job well.



5. Principle 2 - Independence

Each service should be able to

- Operate Independently
- Evolve Independently
- Be Changed Independently

- A **change in one service should not require a change in another service**

For example:

```
Order Service
     ↓
Change order calculation
     ↓
Deploy Order Service
```

You shouldn't have to redeploy:

❌ User Service
❌ Product Service
❌ Payment Service
❌ Inventory Service

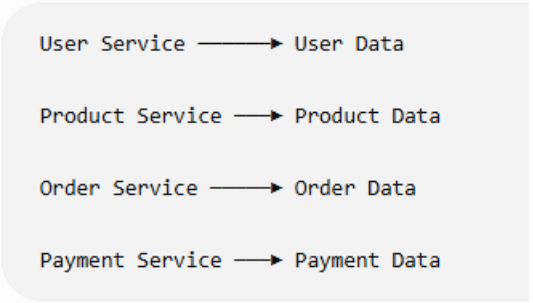
just because the Order Service changed.

This is one of the major improvements over monolithic architecture.

6. Principle 3 - Decentralization

- In microservice architecture, **services are decentralized**
- The services can manage their own
  - Database
  - Data
  - Data Model
- This gives each **service greater autonomy** in choosing how its data is managed.

Conceptually:



7. Principle 4 – Failure Isolation

Suppose the Payment Service fails:

```
Payment Service
      ❌
```

It should **not automatically bring the entire application down**.

Ideally:

```
User Service       ✅
Product Service    ✅
Inventory Service  ✅
Order Service      ✅
Payment Service    ❌
```

The rest of the system can continue functioning to whatever extent the business flow allows.

Remember

> **Failure of one service should be isolated as much as possible from the rest of the system.**

This is called **failure isolation**.

8. Principle 5 – Continuous Delivery & Deployment

- Each microservice should be independently deployable
- Suppose we make a change to the Order Service

```
Order Service
     ↓
Make change
     ↓
Build
     ↓
Test
     ↓
Deploy Order Service
```

You don't necessarily need to redeploy:

```
User Service
Product Service
Payment Service
Inventory Service
```

This enables teams to:

- Release changes more frequently
- Deploy services independently
- Reduce the scope of each release
- Work with greater autonomy

9. Why Do We Need Microservices?

Problems with monolith:

```
Monolith
   ↓
Tight coupling
   ↓
Hard to change independently
   ↓
Hard to deploy independently
   ↓
Hard to scale independently
   ↓
Large and complex codebase
```

Microservices attempt to address these challenges through:

```
Microservices
      ↓
Small services
      ↓
Single responsibility
      ↓
Independent development
      ↓
Independent deployment
      ↓
Independent scaling
      ↓
Failure isolation
```

10. Most Important Principles to Remember

Principle	Meaning
Single Responsibility	Each service has one primary business responsibility
Independence	Services can operate and evolve independently
Decentralization	Services manage their own data/data models
Failure Isolation	Failure of one service shouldn't bring down the whole system
Continuous Delivery/Deployment	Services can be updated and deployed independently